

UNIVERSIDADE PAULISTA – UNIP EaD

Projeto Integrado Multidisciplinar

**Curso Superior de Tecnologia em
Análise e Desenvolvimento de Sistemas**

Yasmine Lima Santana - 2526592

Geovanna Félix Monteiro - 2521699

Luan Lopes Bezerra - 2503460

João Pedro Carneiro da Silva - 2540975

Vitória Freitas - 2523279

Sofia de Castro Ongaro - 2528018

Ecossistema web para clínicas populares

Santana de Parnaíba, SP

2026

Yasmine Lima Santana - 2526592

Geovanna Félix Monteiro - 2521699

Luan Lopes Bezerra - 2503460

João Pedro Carneiro da Silva - 2540975

Vitória Freitas - 2523279

Sofia de Castro Ongaro - 2528018

Ecossistema web para clínicas populares

Projeto Integrado Multidisciplinar em Análise e Desenvolvimento de Sistemas

Projeto Integrado Multidisciplinar para
obtenção do título de tecnólogo em
Análise e Desenvolvimento de Sistemas,
apresentado à Universidade Paulista –
UNIP EaD.

Orientador (a): Tarcísio Peres

Santana de Parnaíba, SP

RESUMO

Nesta monografia documentamos a construção de um ecossistema web para clínicas populares. O desenvolvimento do sistema clínico foi baseado em 4 disciplinas interdisciplinares, essas sendo: engenharia de software ágil aplicada, machine learning, UI e UX design, modelagem de banco de dados e nosql.

Os problemas que a disciplina engenharia de software ágil aplicada teve o propósito de resolver foram: definição dos requisitos que a plataforma terá que cumprir, a criação de módulos do sistema, a metodologia ágil utilizada no projeto, o planejamento de testes e a qualidade de software que o sistema tem o comprometimento de atingir.

Em machine learning construímos dois pipelines de algoritmos de aprendizado de máquina, o primeiro algoritmo foi para regressão e o segundo para classificação. O de regressão utilizou o Random Forest Regressor, com a finalidade de prever o tempo de espera de um paciente e para classificação foi escolhido para Random Forest Classifier, com o objetivo de classificar o histórico de atraso de pacientes.

Na disciplina de UX e UI Design foi feita pesquisa exploratória para entender as necessidades dos usuários dentro de uma clínica popular, e para auxiliar na compreensão dessas necessidades foi construído personas e jornadas de usuários para identificação preventiva de possíveis gargalos durante a utilização do usuário na plataforma.

Com modelagem de dados e nosql criamos as principais entidades do banco de dados (SQL Server), junto com o relacionamento dessas entidades e o script da criação do banco, das tabelas e inserção de dados dentro delas.

Em suma, o que almejamos ao realizar a integração dessas disciplinas, é ter alcançado o objetivo de construir uma plataforma completa para ajudar e auxiliar o trabalho em uma clínica popular, facilitando a rotina, tanto dos profissionais da área quanto as dos clientes que utilizam os seus serviços.

Palavras-chaves: clínica popular, machine learning, engenharia de software, UI e UX, modelagem de dados.

ABSTRACT

In this monograph, we document the construction of a web ecosystem for popular clinics. The development of the clinical system was based on 4 interdisciplinary disciplines: applied agile software engineering, machine learning, UI and UX design, database modeling, and NoSQL.

The problems that the applied agile software engineering discipline aimed to solve were: defining the requirements that the platform will have to meet, creating system modules, the agile methodology used in the project, test planning, and the software quality that the system is committed to achieving.

In machine learning, we built two machine learning algorithm pipelines; the first algorithm was for regression and the second for classification. The regression algorithm used Random Forest Regressor to predict a patient's waiting time, and Random Forest Classifier was chosen for classification to classify patients' delay history.

In the UX and UI Design course, exploratory research was conducted to understand the needs of users within a low-cost clinic. To aid in understanding these needs, personas and user journeys were created to proactively identify potential bottlenecks during user interaction with the platform.

Using data modeling and NoSQL, we created the main database entities (SQL Server), along with the relationships between these entities and the script for creating the database, tables, and inserting data into them.

In short, our goal in integrating these disciplines is to build a complete platform to assist and support the work in a low-cost clinic, facilitating the routine of both professionals in the field and clients who use their services.

Keywords: low-cost clinic, machine learning, software engineering, UI and UX, data modeling.

SUMÁRIO

1. INTRODUÇÃO.....	6
2. ENGENHARIA DE SOFTWARE ÁGIL APLICADA.....	7
2.1 Visão geral do sistema.....	7
2.2 Módulos do sistema.....	7
2.3 Requisitos funcionais.....	8
2.4 Requisitos não funcionais.....	9
2.5 Metodologia ágil.....	9
2.6 User Stories.....	10
2.7 Product Backlog.....	11
2.8 Planejamento de sprints.....	12
2.9 Plano de testes.....	13
2.10 Qualidade de software.....	14
3. MACHINE LEARNING - CRIAÇÃO DE ALGORITMO DE REGRESSÃO E CLASSIFICAÇÃO.....	15
3.1 Dicionário de dados.....	15
3.2 Pipeline do algoritmo de regressão.....	17
3.3 Pipeline do algoritmo de classificação.....	20
3.4 Resultados das métricas.....	23
4. UX E UI DESIGN.....	24
4.1 Pesquisa Exploratória.....	24
4.2 Construção das Personas.....	25
4.3 Jornada do Usuário.....	26
4.4 Fluxo de Navegação.....	28
4.5 Desenvolvimento dos Wireframes.....	30
4.6 Desenvolvimento do Protótipo Iterativo.....	32
4.7 Testes de Usabilidade.....	34
4.8 Resultados Obtidos.....	36
5. MODELAGEM DE BANCO DE DADOS E NOSQL.....	37

5.1 Entidades do banco de dados.....	37
5.1.1 Entidade Pacientes.....	37
Atributos:.....	37
5.1.2 Entidade Profissionais.....	37
Atributos:.....	37
5.1.3 Entidade Agendamentos.....	38
Atributos:.....	38
5.1.4 Entidade Atendimentos.....	38
Atributos:.....	38
5.2 Relacionamentos entre as tabelas.....	38
5.3 Regras de Negócio.....	38
5.4 Script SQL (DDL).....	39
5.5 Inserção de Dados (DML).....	40
5.6 Consultas.....	41
5.6.1 Listar pacientes.....	41
5.6.2 Consultas agendadas.....	41
5.6.3 Histórico de atendimento de um paciente.....	41
6. CONCLUSÃO.....	42
REFERÊNCIAS.....	43

1. INTRODUÇÃO

Na rotina de uma clínica popular a diversos desafios como o atraso de consultas, dados desorganizados, filas lotadas em determinados horários, tendo isso em mente esse trabalho propôs a construção de um ecossistema web para clínicas populares, com a finalidade de resolver alguns dos problemas que possam ter dentro de uma, como os citados anteriormente.

Para tal objetivo, utilizamos quatro disciplinas que cuidam de uma parte do sistema independentemente, mas funcionam como um ecossistema quando integradas, essas disciplinas são: engenharia de software ágil aplicada, modelagem de banco de dados e nosql, UI e UX design e machine learning.

Com engenharia de software ágil aplicada foi determinado os requisitos funcionais e não funcionais da plataforma, junto com a metodologia adotada para o desenvolvimento e o seu planejamento. Em machine learning criamos os pipelines dos algoritmos de aprendizado de máquina para classificação e regressão. Na sessão de UI e UX design foi construído pesquisas exploratória, persona e jornada de usuário com a finalidade de criar um sistema intuitivo, com uma boa interface que atenda às exigências do usuário final. Por fim, com a disciplina de modelagem de banco de dados e nosql foi descrito as entidades do banco de dados, os seus relacionamentos, atributos e o script SQL com a criação delas, das tabelas e do banco de dados.

Desse modo o nosso projeto tem o intuito de desenvolver um software que atenda os anseios dos usuários e resolva os obstáculos que ele se propôs a resolver, tendo em vista as necessidades dos profissionais da saúde e dos seus clientes dentro do cenário que o sistema vai ser utilizado.

2. ENGENHARIA DE SOFTWARE ÁGIL APLICADA

2.1 Visão geral do sistema

O presente projeto relata o desenvolvimento de um sistema web, direcionado a clínicas populares, com ênfase na gestão dos prontuários eletrônicos e na organização de forma inteligente de filas de espera.

O sistema foi pensado com objetivo de solucionar problemas comuns encontrados em clínicas de menor custo, sendo no geral os mais presentes no cotidiano: atrasos nos atendimentos, filas longas, perda de informações médicas e a falta de organização administrativa.

Destarte, a plataforma permitirá que recepcionistas, médicos e pacientes utilizem funcionalidades específicas dentro do sistema, de acordo com suas funções e necessidades. Resultando em uma maior eficiência operacional e experiências gratificantes para o paciente.

Sendo assim, as principais funcionalidades pretendidas para a realização do projeto são: o cadastro de pacientes, agendamentos de consultas, histórico de atendimentos, relatório administrativo, opções para remarcação/cancelamentos e gerenciamento de filas em tempo real. Desta forma, o sistema será acessado através de navegadores Web, permitindo a visualização através de smartphones, tablets e computadores.

2.2 Módulos do sistema

Para melhor organização do projeto, o sistema foi dividido em módulos contribuindo também para o gerenciamento das funcionalidades. Para a execução dos módulos abaixo, foram estudados os sistemas de clínicas, como: Doutoraria, sistema de UBS de Barueri e diversas clínicas com o mesmo público-alvo, consideravelmente concorrentes diretas.

- Módulo de Cadastro: função de armazenamento das informações dos pacientes, profissionais da saúde e usuário do sistema;
- Módulo de Agendamento: opções de agendar, remarcar e cancelar

futuras consultas;

- Módulo de Fila Inteligente: gerenciar automaticamente a fila de espera, organizando por prioridades e por possíveis cancelamentos/remarcações;
- Módulo de Prontuário Eletrônico: Registra o histórico clínico do paciente, consultas realizadas, alergias, prescrições médicas e prescrições.

2.3 Requisitos funcionais

Os requisitos funcionais correspondem ao conjunto de funcionalidades que o sistema deverá ser capaz de executar, descrevendo os comportamentos e serviços esperados sob determinadas condições de operação. Conforme apresentado no Quadro 1, foram identificados sete requisitos funcionais para o sistema proposto.

Tabela 1 - Requisitos Funcionais

Código	Requisito Funcional
RF01	O sistema deverá permitir o cadastro de pacientes
RF02	O sistema deverá permitir o login de usuários
RF03	O sistema deverá gerar filas inteligentes
RF04	O sistema deverá exibir o tempo estimado de espera
RF05	O sistema deverá permitir o agendamento de consultas
RF06	O sistema deverá armazenar prontuários médicos
RF07	O sistema deverá emitir relatórios administrativos

Fonte: autoria própria

2.4 Requisitos não funcionais

Os requisitos não funcionais estabelecem os critérios de qualidade, desempenho e segurança que o sistema deverá atender, não se relacionando diretamente às funcionalidades, mas sim às condições sob as quais essas funcionalidades devem operar. O Quadro 2 apresenta os requisitos não funcionais definidos para o sistema.

Tabela 2 - Requisitos Não Funcionais

Código	Requisitos Não Funcionais
RNF01	O sistema deverá possuir interface intuitiva
RNF02	O sistema deverá responder em até 3 (três) segundos
RNF03	O sistema deverá garantir a segurança dos dados
RNF04	O sistema deverá possuir mecanismo de autenticação de usuários
RNF05	O sistema deverá estar disponível 24 (vinte e quatro) horas por dia
RNF06	O sistema deverá estar em conformidade com os princípios da Lei Geral de Proteção de Dados (LGPD)

Fonte: autoria própria

2.5 Metodologia ágil

Para o desenvolvimento do presente projeto, adotou-se a metodologia ágil Scrum, amplamente reconhecida na literatura por sua capacidade de organização incremental e adaptação contínua às mudanças e às necessidades emergentes ao longo do ciclo de desenvolvimento de software (SCHWABER; SUTHERLAND, 2020).

O Scrum fundamenta-se na divisão do projeto em ciclos iterativos denominados *sprints*, possibilitando entregas parciais e funcionais ao término de cada iteração. Essa abordagem favorece o controle sistemático das tarefas, a priorização

2.6 User Stories

As *User Stories*, ou histórias de usuário, constituem uma técnica amplamente empregada no desenvolvimento ágil de software para representar, de forma simples e centrada no usuário, as necessidades e expectativas dos diferentes perfis que interagirão com o sistema (COHN, 2004). Cada história é estruturada a partir da perspectiva de um ator específico, descrevendo o que ele deseja realizar e qual benefício espera obter com determinada funcionalidade.

No contexto do presente projeto, as histórias de usuário foram elaboradas com base nos perfis identificados durante o levantamento de requisitos, sendo eles: recepcionista, médico, paciente e administrador. O Quadro 3 apresenta as histórias de usuário definidas para o sistema.

Tabela 3 - User Story

ID	User Story
US01	Como recepcionista, desejo cadastrar pacientes para agilizar os atendimentos
US02	Como médico, desejo acessar o prontuário eletrônico para consultar o histórico do paciente
US03	Como paciente, desejo acompanhar minha posição na fila para saber o tempo estimado de espera
US04	Como administrador, desejo gerar relatórios para analisar os

	atendimentos realizados
--	-------------------------

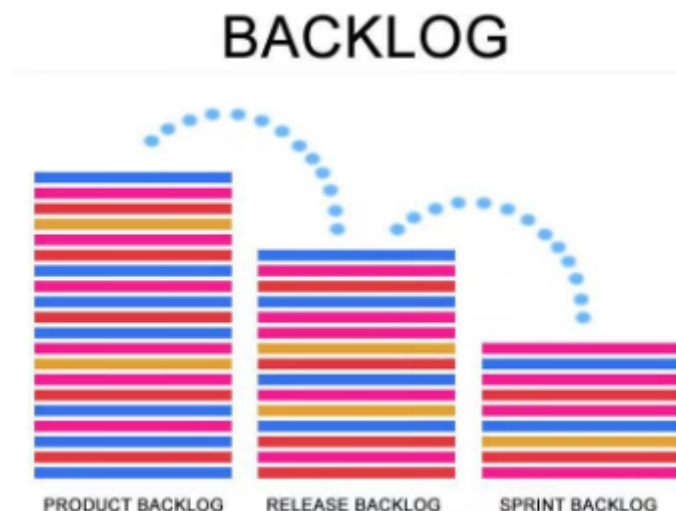
Fonte: autoria própria

2.7 Product Backlog

O *Product Backlog* constitui um artefato central da metodologia Scrum, correspondendo a uma lista ordenada e dinâmica de todas as funcionalidades, melhorias e requisitos necessários ao desenvolvimento do produto (SCHWABER; SUTHERLAND, 2020). Sua organização por nível de prioridade permite que a equipe de desenvolvimento concentre esforços nas entregas de maior valor ao negócio nas fases iniciais do projeto.

A figura abaixo ilustra a estrutura hierárquica do backlog no contexto do Scrum, evidenciando a relação entre o *Product Backlog*, o *Release Backlog* e o *Sprint Backlog*, demonstrando como os itens são progressivamente refinados e selecionados para cada ciclo de desenvolvimento.

Figura 1 - Imagem backlog



Fonte: t2 informatic

No presente projeto, os itens do *Product Backlog* foram classificados em três

níveis de prioridade — alta, média e baixa —, levando em consideração a criticidade de cada funcionalidade para o funcionamento essencial do sistema. O Quadro 4 apresenta o *Product Backlog* definido para o projeto.

Tabela 4 - Funcionalidades

Prioridade	Funcionalidade
Alta	Cadastro de pacientes
Alta	Sistema de login
Alta	Gestão de filas
Média	Relatórios administrativos
Média	Histórico médico
Baixa	Painel estatístico

Fonte: autoria própria

2.8 Planejamento de sprints

O planejamento das *sprints* representa uma das cerimônias mais relevantes da metodologia Scrum, na qual a equipe de desenvolvimento seleciona, a partir do *Product Backlog*, os itens que serão implementados ao longo de cada ciclo iterativo.

Para Pressman e Maxim (2016), a divisão do desenvolvimento em incrementos curtos e bem delimitados favorece a detecção precoce de falhas e a adaptação contínua aos requisitos do projeto, reduzindo significativamente os riscos associados ao processo de desenvolvimento.

Cada *sprint* possui escopo definido, duração preestabelecida e uma entrega

incremental ao término do ciclo. Para o presente projeto, o desenvolvimento foi estruturado em quatro *sprints*, organizadas de forma a priorizar as funcionalidades essenciais nas fases iniciais e reservar as etapas de validação e refinamento para os ciclos finais. O Quadro 5 apresenta a distribuição das atividades por *sprint*.

Tabela 5 - Planejamento de Sprints

Sprint	Funcionalidade desenvolvida
1	Cadastro de usuários; Sistema de login; Estrutura inicial do banco de dados
2	Agendamento de consultas; Gerenciamento de filas
3	Prontuário eletrônico; Relatórios administrativos
4	Testes e correções; Ajustes finais

Fonte: autoria própria

2.9 Plano de testes

No presente projeto, foram definidas três categorias de teste: testes unitários, voltados à verificação de componentes individuais do sistema; testes de integração, destinados a validar a comunicação entre os diferentes módulos; e testes de aceitação, orientados à confirmação de que o sistema atende às necessidades dos usuários finais, conforme as histórias de usuário previamente definidas. Para Sommerville (2019), os testes de aceitação são particularmente relevantes em projetos desenvolvidos sob metodologias ágeis, pois envolvem diretamente os usuários na validação das entregas incrementais.

A tabela abaixo apresenta os cenários de teste definidos para o sistema, com a descrição de cada caso e o respectivo resultado esperado:

Tabela 6 - Planos de testes

ID	Cenário de Teste	Resultado Esperado
CT01	Login com credenciais válidas	Acesso ao sistema permitido
CT02	Inserção de CPF inválido	Exibição de mensagem de erro ao usuário
CT03	Envio de cadastro incompleto	Solicitação de preenchimento dos campos obrigatórios
CT04	Agendamento de consulta	Consulta registrada com sucesso no sistema

Fonte: autoria própria

2.10 Qualidade de software

A qualidade de software está relacionada à capacidade do sistema atender corretamente aos requisitos definidos, garantindo bom funcionamento, desempenho, segurança e facilidade de uso (PRESSMAN; MAXIM, 2016). Para isso, o sistema foi desenvolvido considerando características importantes definidas pela norma ISO/IEC 25010 (2011), como usabilidade, eficiência, segurança, confiabilidade, acessibilidade e manutenibilidade.

A usabilidade foi aplicada por meio de uma interface intuitiva e de fácil utilização para diferentes perfis de usuários. A eficiência busca garantir respostas rápidas às ações realizadas no sistema. Já a segurança envolve mecanismos de autenticação e controle de acesso para proteger os dados dos pacientes e restringir funcionalidades conforme o perfil do usuário.

Além disso, o sistema foi projetado para operar de forma estável e contínua, assegurando confiabilidade e disponibilidade. Também foram considerados aspectos de acessibilidade e facilidade de manutenção, permitindo futuras atualizações e correções no sistema.

Por fim, o desenvolvimento seguiu os princípios da Lei Geral de Proteção de Dados (LGPD), garantindo maior proteção às informações sensíveis dos pacientes e reduzindo riscos de acessos não autorizados.

3. MACHINE LEARNING - CRIAÇÃO DE ALGORITMO DE REGRESSÃO E CLASSIFICAÇÃO

Para a disciplina de machine learning criamos dois pipeline, um que inclui um algoritmo de regressão para prever o tempo de espera de um paciente, e outro com algoritmo de classificação para classificar o histórico de atraso de pacientes, se probabilidade dele se atrasar é baixa, média ou elevada. Para a construção desses pipelines utilizamos as seguintes bibliotecas do Python: sklearn, numpy, pandas e matplotlib.

O algoritmo de regressão utilizado foi o Random Forest Regressor e para classificação foi o Random Forest Classifier. Random Forest é um método de aprendizagem que combina múltiplas árvores de decisão para produzir respostas com acurácia alta e estáveis, usado tanto para regressão e classificação. As previsões de regressão são obtidas pela média dos resultados das diversas árvores, as previsões de classificação é a qual foi escolhida pela maioria das árvores.

3.1 Dicionário de dados

Tabela 7 - dicionário de dados

Campo	Descrição	Tipo
horario	Horário da consulta	Hora

dia_semana	Dia da consulta	Texto
tipo_consulta	Especialidade da consulta	Texto
pacientes_na_fila_antes	Quantidade de pacientes aguardando antes	Numérico
pacientes_agendados_no_mesmo_horario	Total de pacientes marcados no mesmo horário	Numérico
tempo_medio_consulta_tipo_min	Tempo médio da consulta por especialidade (min)	Numérico
medico_habito_atraso	Média de atraso do médico (min)	Numérico
horario_pico	Indica se o horário é de pico (sim/não)	Texto
tempo_de_espera_min	Tempo de espera do paciente	Numérico
idade	Idade do paciente	Numérico
consulta	Tipo/especialidade da consulta	Texto

distacia_local_de_partida_e_consultorio_km	Distância entre origem e consultório (km)	Numérico
tipo_de_transporte	Meio de transporte utilizado	Texto
horario_da_consulta	Horário da consulta	Hora
PcD	Indica se o paciente é Pessoa com Deficiência	Texto
histórico_de_atrasos	Histórico de atrasos do paciente (baixo, médio, elevado)	Texto

Fonte: autoria própria

3.2 Pipeline do algoritmo de regressão

```
#Previsão de tempo estimado de espera

df = pd.read_csv('/content/bd_tmp_espera2.csv')

df.head(5)

print(f"\nTotal de registros: {len(df)}")

print(f"\nColunas: {list(df.columns)}")

print(df.isnull().sum())

print(df.head(10))

print(df['tempo_de_espera_min'].describe())
```

```

x = df.drop('tempo_de_espera_min', axis=1)

y = df['tempo_de_espera_min'].values

preprocessor = ColumnTransformer(

    transformers=[

        ('cat', Pipeline(steps=[

            ('imputer', SimpleImputer(strategy='most_frequent')),

            ('onehot', OneHotEncoder(handle_unknown='ignore'))

        ]), ['horario', 'dia_semana', 'tipo_consulta', 'horario_pico']),

        ('num', Pipeline(steps=[

            ('imputer', SimpleImputer(strategy='mean')),

            ('scaler', StandardScaler())

        ]), ['pacientes_na_fila_antes', 'pacientes_agendados_no_mesmo_horario',

            'tempo_medio_consulta_tipo_min', 'medico_habito_atraso'])

    ],

    remainder='drop'

)

X_transform = preprocessor.fit_transform(x)

X_Train, X_Test, y_Train, y_Test = train_test_split(X_transform, y, test_size=0.3,
random_state=42)

paramgrid = {

    'n_estimators': [100, 200, 300, 400, 500],

    'max_depth': [None, 10, 20, 30],

```

```

'min_samples_split': [2, 5, 10],

'min_samples_leaf': [1, 2, 4],

'max_features': [1.0, 'sqrt', 'log2', None],

'bootstrap': [True, False]
}

rfr = RandomForestRegressor()

random_search = RandomizedSearchCV(

    estimator=rfr,

    param_distributions=paramgrid,

    n_iter=10,

    cv=5,

    verbose=2,

    scoring="neg_mean_squared_error",

    random_state=42,

    n_jobs=-1

)

random_search.fit(X_Train, y_Train)

best_params = random_search.best_params_

random_forest_regressor = RandomForestRegressor(**best_params)

random_forest_regressor.fit(X_Train, y_Train)

test_predic = random_forest_regressor.predict(X_Test)

```

```

mse = mean_squared_error(y_Test, test_predic)

r2 = r2_score(y_Test, test_predic)

mae = mean_absolute_error(y_Test, test_predic)

z = np.polyfit(y_Test, test_predic, 1)

p = np.poly1d(z)

plt.figure(figsize=(10, 6))

plt.scatter(y_Test, test_predic, alpha=0.6)

plt.plot(y_Test, p(y_Test), color='red', linestyle='--')

plt.xlabel('Valores Reais')

plt.ylabel('Valores Previstos')

df_to_predict = pd.read_csv('/content/bd_tmp_espera_to_predict.csv')

df_to_predict_transform = preprocessor.transform(df_to_predict)

predict = random_forest_regressor.predict(df_to_predict_transform)

df_to_predict['tempo_de_espera_min'] = predict

df_to_predict.head(5)

```

3.3 Pipeline do algoritmo de classificação

```

df_classifier = pd.read_csv('/content/data_to_train_classifier.csv')

preprocessor_classifier = ColumnTransformer(

    transformers=[

        ('cat', Pipeline(steps=[

```

```

        ('onehot', OneHotEncoder(handle_unknown='ignore'))

        ]), ['consulta', 'tipo_de_transporte', 'dia_semana', 'horario_da_consulta',
'horario_pico', 'PcD']

    ),

    ('num', Pipeline(steps=[

        ('scaler', StandardScaler())

        ]), ['idade', 'distacia_local_de_partida_e_consultorio_km'])

    ],

    remainder='drop'

)

x = df_classifier.drop('histórico_de_atrasos', axis=1)

y = df_classifier['histórico_de_atrasos'].values

label_encoder = LabelEncoder()

X_transformed = preprocessor_classifier.fit_transform(x)

Y_transformed = label_encoder.fit_transform(y)

smote = SMOTE(sampling_strategy='auto', random_state=42)

X_balanced, y_balanced = smote.fit_resample(X_transformed, Y_transformed)

X_train, X_test, y_train, y_test = train_test_split(X_balanced, y_balanced,
test_size=0.3, random_state=42)

rfc = RandomForestClassifier()

parameters = {

    'n_estimators': [100, 200, 300, 400, 500],

```

```

'max_depth': [10, 20, 30],

'min_samples_leaf': [1, 2, 4, 5],

'min_samples_split': [2, 3, 4, 5, 6, 7, 8, 9, 10],

'max_features': [1.0, 'sqrt', 'log2', None],

'max_samples': [0.5, 0.6, 0.7, 0.8, 0.9, 1.0],

'bootstrap': [True, False]
}

random_searchCV = RandomizedSearchCV(

    estimator=rfc,

    param_distributions=parameters,

    n_iter=10,

    cv=5,

    verbose=2,

    scoring="neg_mean_squared_error",

    random_state=42,

    n_jobs=-1

)

random_searchCV.fit(X_train, y_train)

best_params_to_rfc = random_searchCV.best_params_

Random_Forest_Classifier = RandomForestClassifier(**best_params_to_rfc)

Random_Forest_Classifier.fit(X_train, y_train)

```

```

y_pred = Random_Forest_Classifier.predict(X_test)

conf_matrix = confusion_matrix(y_test, y_pred)

f1Score = f1_score(y_test, y_pred, average='weighted')

accuracy = accuracy_score(y_test, y_pred)

df_classifier_predict = pd.read_csv('/content/data_to_classifier_classify.csv')

classify_predict = preprocessor_classifier.transform(df_classifier_predict)

predict_class = Random_Forest_Classifier.predict(classify_predict)

predictions_string = label_encoder.inverse_transform(predict_class)

df_classifier_predict['histórico_de_atrasos'] = predictions_string

df_classifier_predict.head(50)

```

3.4 Resultados das métricas

Tabela 8 - Métricas

Métrica	Resultado
Mse (Mean Square Error)	2566.58
R2 (r2 Score)	0.92
Mae (Mean Absolute Error)	39.80

Conf_matrix (Confusion Matrix)	[[40 18 14] [16 36 18] [20 20 31]]
f1Score (f1 score)	0.50
accuracy (accuracy score)	0.50

Fonte: autoria própria

4. UX E UI DESIGN

4.1 Pesquisa Exploratória

Antes do desenvolvimento das interfaces do sistema, foi realizada uma pesquisa exploratória com o objetivo de compreender as necessidades dos principais usuários envolvidos na rotina de uma clínica popular. A pesquisa foi baseada na observação do cenário apresentado no projeto, considerando as dificuldades enfrentadas por pacientes, recepcionistas e profissionais de saúde durante os processos de agendamento, atendimento e consulta de informações.

Durante a análise do contexto, foram identificados alguns problemas recorrentes. Os pacientes possuem pouca visibilidade sobre o tempo de espera e frequentemente não sabem sua posição na fila de atendimento. As recepcionistas precisam controlar simultaneamente agendamentos, encaixes, cancelamentos e atendimentos presenciais, o que aumenta a possibilidade de erros operacionais. Já os profissionais de saúde enfrentam dificuldades para localizar rapidamente o histórico dos pacientes quando as informações estão dispersas ou registradas de forma não padronizada.

Com base nessas observações, foi definida a proposta de um sistema capaz de integrar prontuário eletrônico, gerenciamento de filas e acompanhamento de

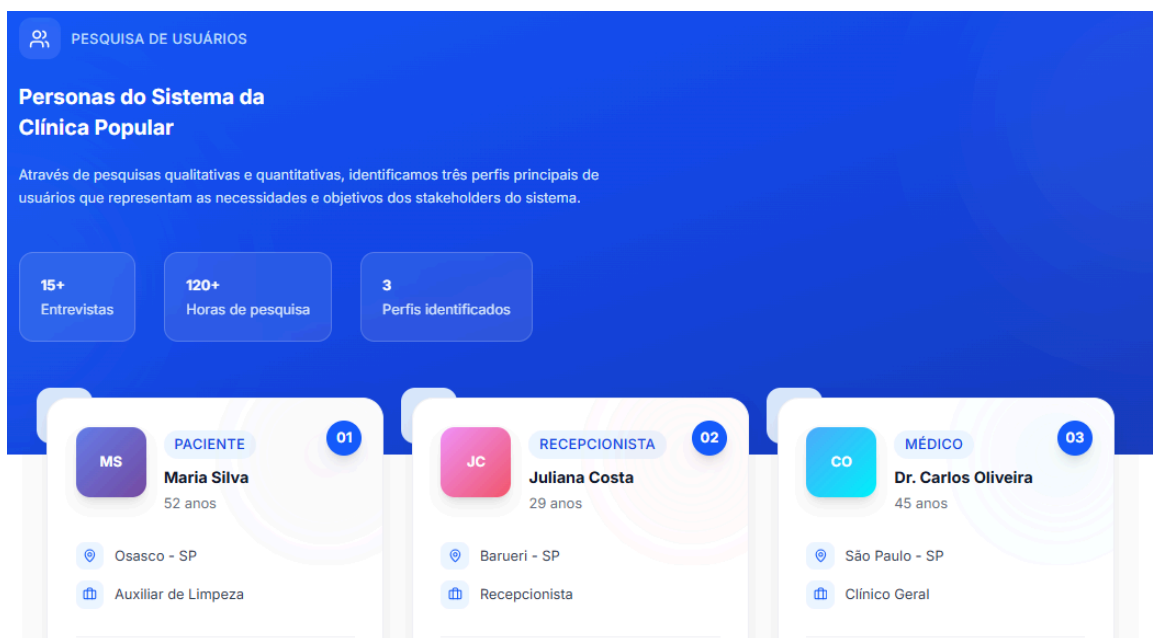
consultas em um único ambiente digital, proporcionando maior organização e melhor experiência para todos os usuários.

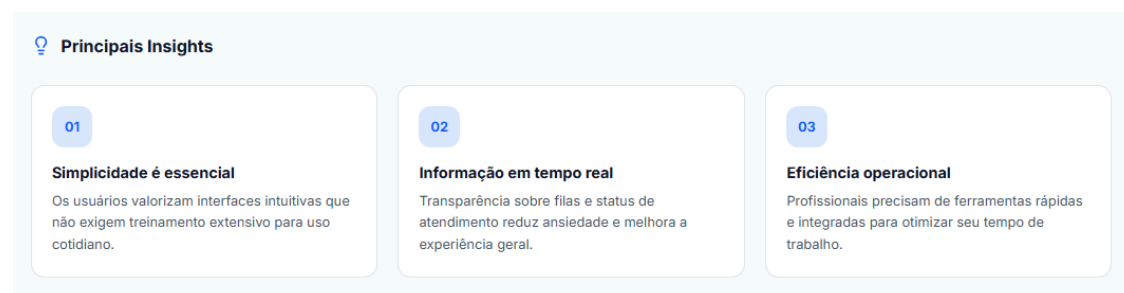
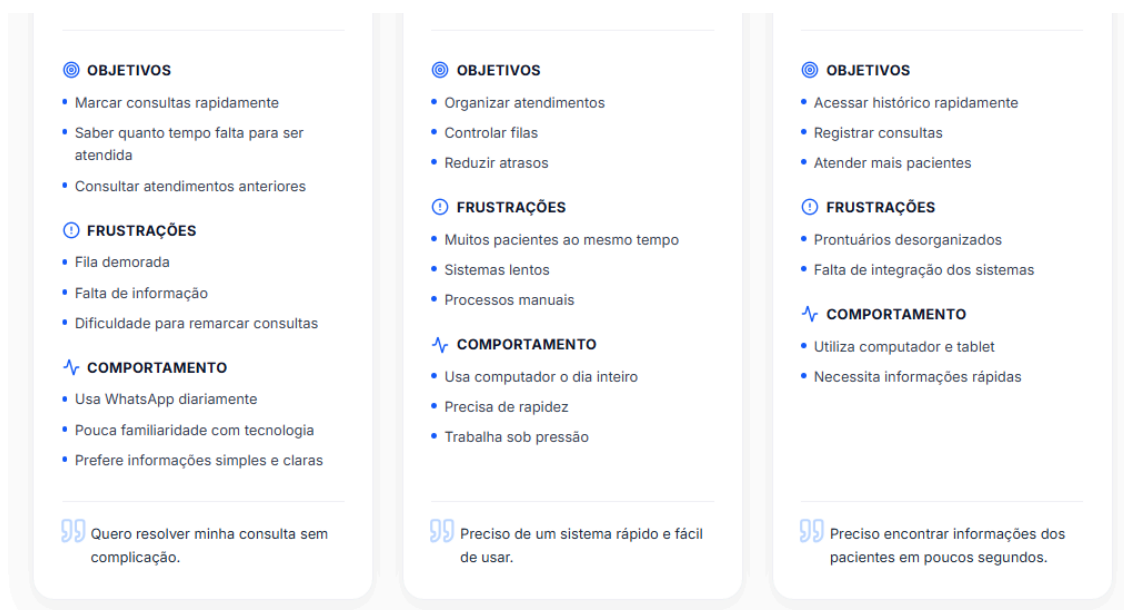
4.2 Construção das Personas

Após a etapa de pesquisa, foram desenvolvidas três personas representando os principais perfis de usuários do sistema. As personas foram criadas para auxiliar na compreensão das necessidades, objetivos e dificuldades de cada grupo de usuários, servindo como referência durante todo o processo de design.

As personas representam um paciente típico de clínica popular, uma recepcionista responsável pela gestão dos atendimentos e um profissional de saúde que utiliza o sistema para acessar prontuários e registrar consultas.

Figura 2 - Persona





Fonte: Autoria própria

A análise das personas permitiu identificar características importantes para o desenvolvimento da solução. Os pacientes demonstraram necessidade de informações claras e simples sobre consultas e filas. As recepcionistas necessitam de agilidade operacional para atender múltiplas demandas simultaneamente. Já os profissionais de saúde precisam acessar informações clínicas de maneira rápida e organizada.

Essas informações serviram de base para a construção das jornadas dos usuários e para a definição das funcionalidades mais importantes do sistema.

4.3 Jornada do Usuário

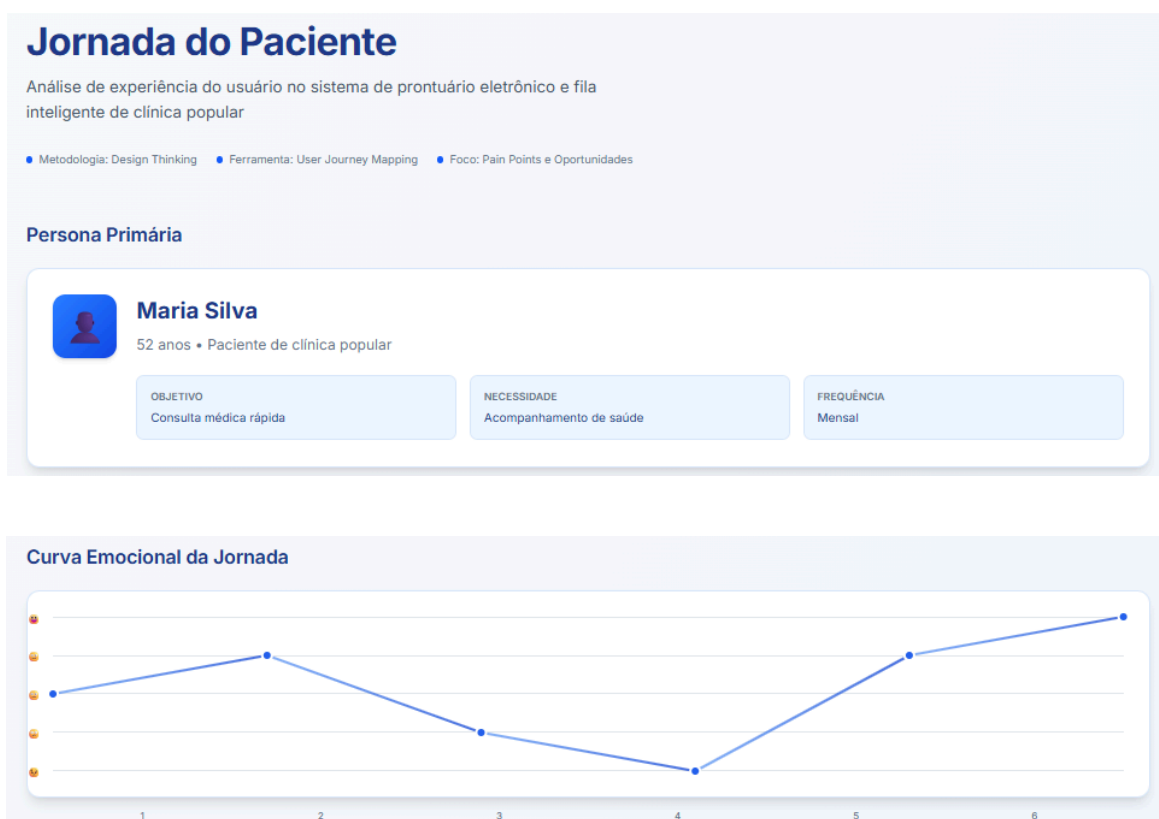
Com as personas definidas, foi desenvolvido o mapa de jornada do usuário com o objetivo de compreender as etapas percorridas pelos pacientes durante sua experiência na clínica.

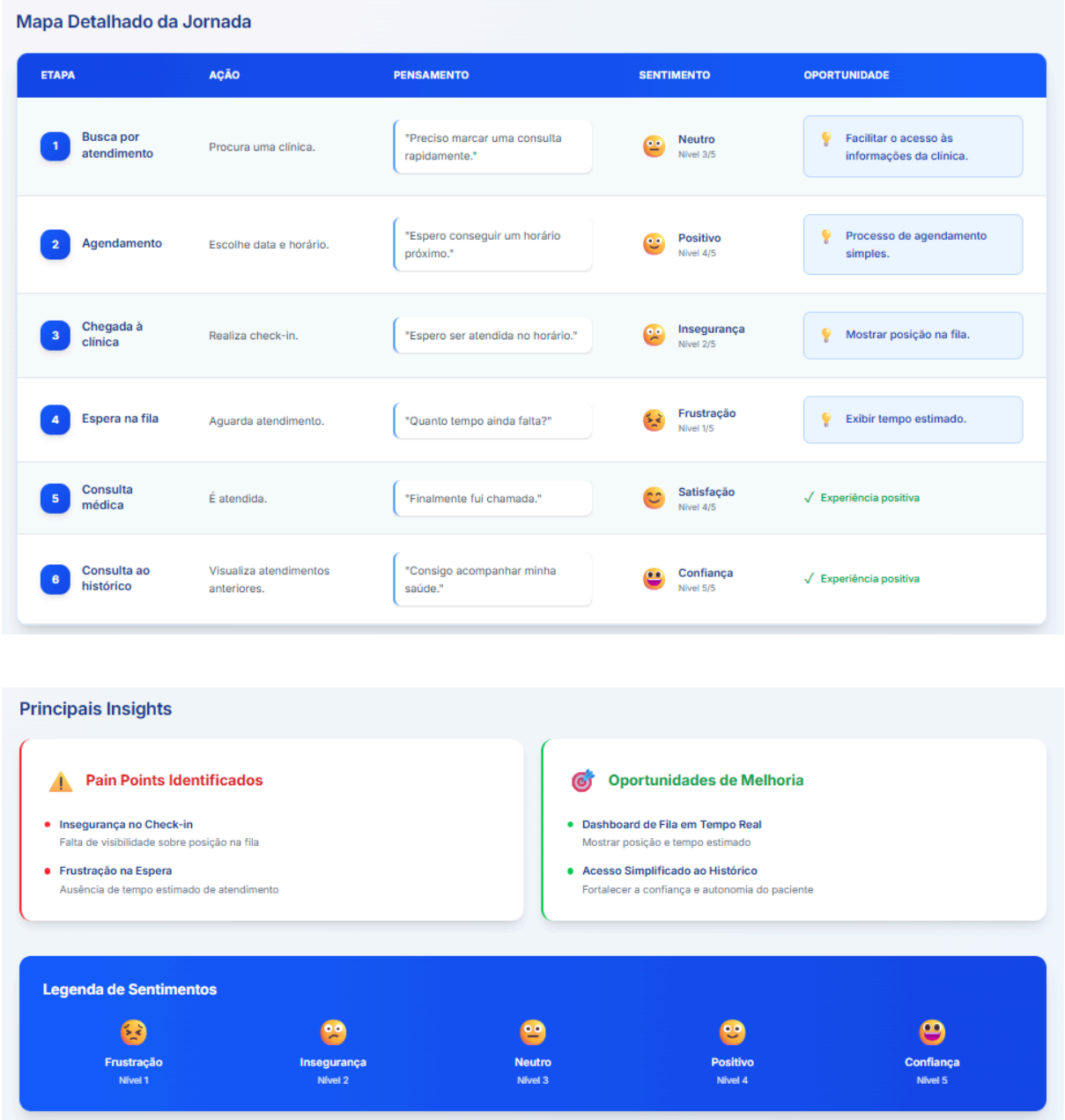
A jornada foi construída considerando desde o momento em que o paciente busca atendimento até a consulta de seu histórico médico após o atendimento realizado.

As etapas identificadas foram:

- Busca por atendimento.
- Agendamento da consulta.
- Chegada à clínica.
- Entrada na fila de espera.
- Atendimento médico.
- Consulta ao histórico de atendimentos.

Figura 3 - Jornada do paciente





Fonte: Autoria própria

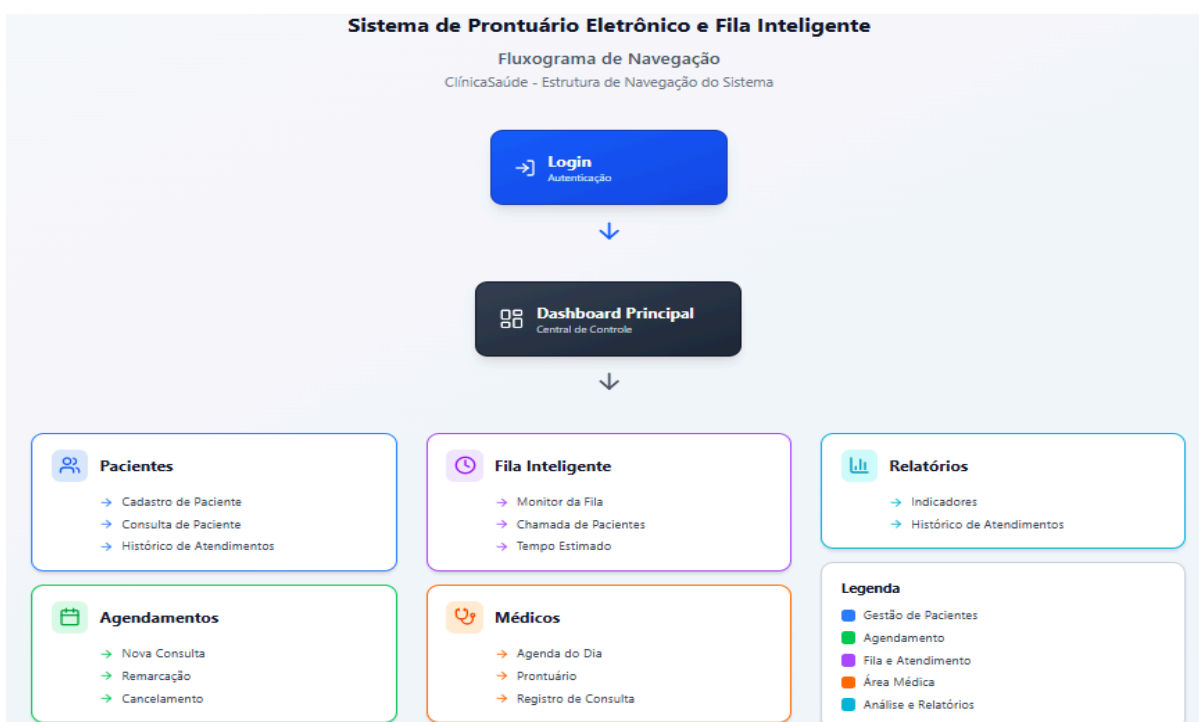
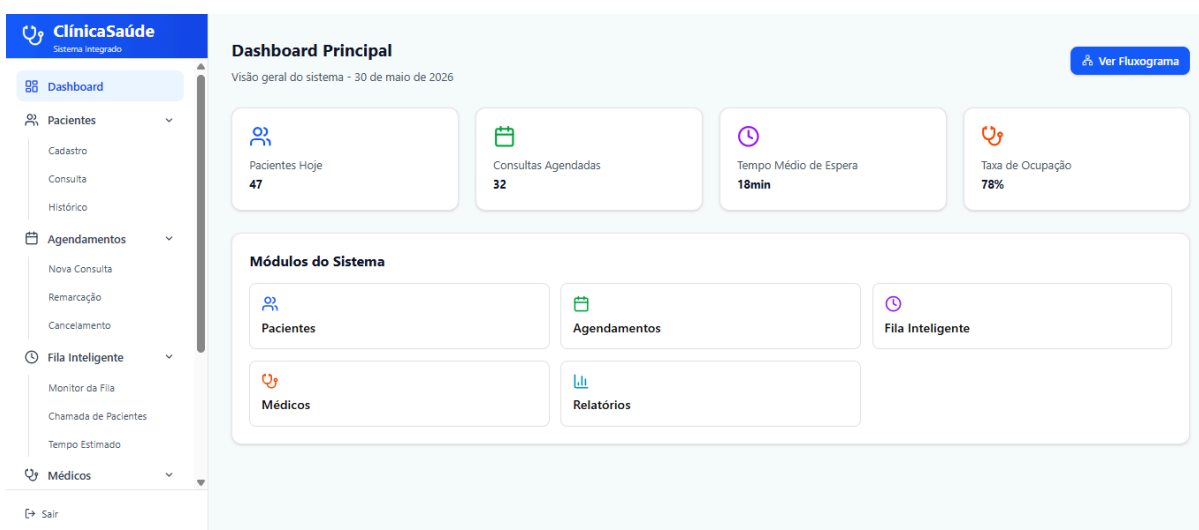
A análise da jornada demonstrou que os principais pontos de dificuldade estão relacionados ao acompanhamento da fila e à falta de informações sobre o tempo estimado de espera. Dessa forma, uma das prioridades do sistema foi fornecer transparência durante o processo de atendimento, reduzindo a ansiedade dos pacientes e melhorando a organização da clínica.

4.4 Fluxo de Navegação

Após o mapeamento da jornada dos usuários, foi elaborado o fluxo de navegação do sistema. O objetivo foi organizar a estrutura das telas e definir o caminho que cada usuário percorrerá para executar suas tarefas.

O fluxo contempla funcionalidades como autenticação de usuários, cadastro de pacientes, agendamento de consultas, gerenciamento de filas, acesso ao prontuário eletrônico e consulta ao histórico de atendimentos.

Figura 4 - Dashboard Principal



Fonte: Autoria própria

A criação do fluxo permitiu identificar a sequência lógica das ações e reduzir etapas desnecessárias, contribuindo para uma navegação mais simples e intuitiva.

4.5 Desenvolvimento dos Wireframes

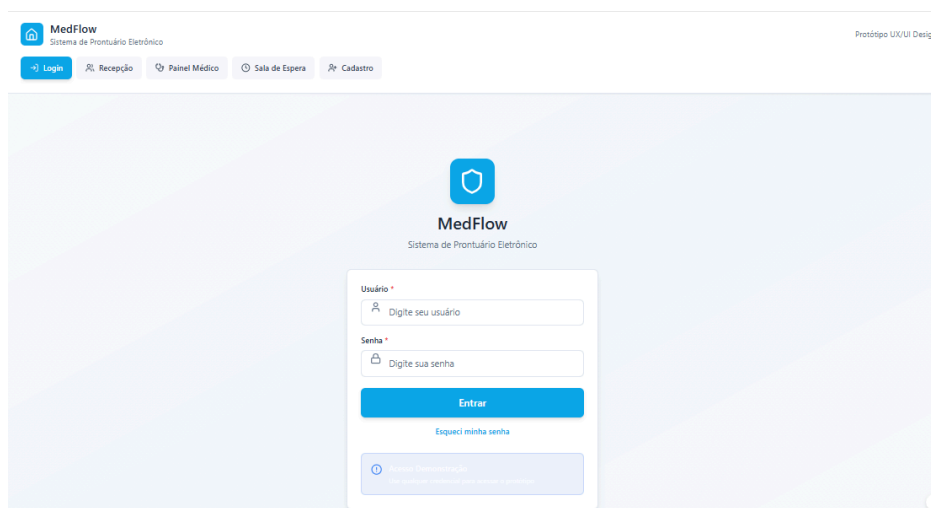
Com a estrutura de navegação definida, foram desenvolvidos wireframes de baixa fidelidade para validar a organização das informações antes da criação do protótipo visual.

Os wireframes foram utilizados para representar a disposição dos elementos nas telas sem preocupação inicial com cores, identidade visual ou aspectos gráficos mais avançados.

As telas projetadas foram:

- Tela de Login.
- Painel da Recepção.
- Painel da Sala de Espera.
- Formulário de Cadastro de Pacientes.

Figura 5 - Wireframe



MedFlow
Sistema de Prontuário Eletrônico

Protótipo UX/UI Design

Login Recepção **Painel Médico** Sala de Espera Cadastro

Painel da Recepção

Gerencie a fila de atendimentos e pacientes

+ Novo Agendamento

Fila Atual
12

Em Atendimento
3

Próximos
9

Lista de Pacientes Aguardando

Q Buscar paciente...

SENHA	PACIENTE	HORARIO	ESPECIALIDADE	STATUS	AÇÕES
1	João Silva	08:30	Clínica Geral	Aguardando	Ver Detalhes
2	Maria Santos Urgente	08:45	Cardiologia	Aguardando	Ver Detalhes
3	Pedro Costa	09:00	Ortopedia	Em Atendimento	Ver Detalhes
4	Ana Oliveira	09:15	Pediatria	Aguardando	Ver Detalhes
5	Carlos Mendes	09:30	Clínica Geral	Aguardando	Ver Detalhes

MedFlow
Sistema de Prontuário Eletrônico

Protótipo UX/UI Design

Login Recepção **Painel Médico** **Sala de Espera** Cadastro

Painel de Chamadas

Acompanhe sua senha e aguarde ser chamado

SENHA ATUAL

A12

Consultório 02

PRÓXIMA SENHA

A13

Aguarde sua vez

Informações do Consultório

Consultório 02

Médico Dr. João Silva

Tempo de Espera

15

minutos (aproximados)

MedFlow
Sistema de Prontuário Eletrônico

Protótipo UX/UI Design

Login Recepção Painel Médico Sala de Espera **Cadastro**

Cadastro de Paciente

Preencha os dados do paciente para realizar o cadastro no sistema

Dados Pessoais

Nome Completo *

CPF *

Data de Nascimento *

Sexo *

Telefone *

Endereço

CEP

Rua *

Número *

Complemento

Bairro *

Fonte: Autoria própria

O uso dos wireframes permitiu validar rapidamente a estrutura das telas e realizar ajustes antes da construção do protótipo interativo.

4.6 Desenvolvimento do Protótipo Interativo

Após a validação dos wireframes, foi desenvolvido um protótipo interativo utilizando a ferramenta Figma.

Durante a construção das interfaces foram aplicados conceitos estudados na disciplina de UX e UI Design, buscando garantir facilidade de uso, clareza visual e acessibilidade.

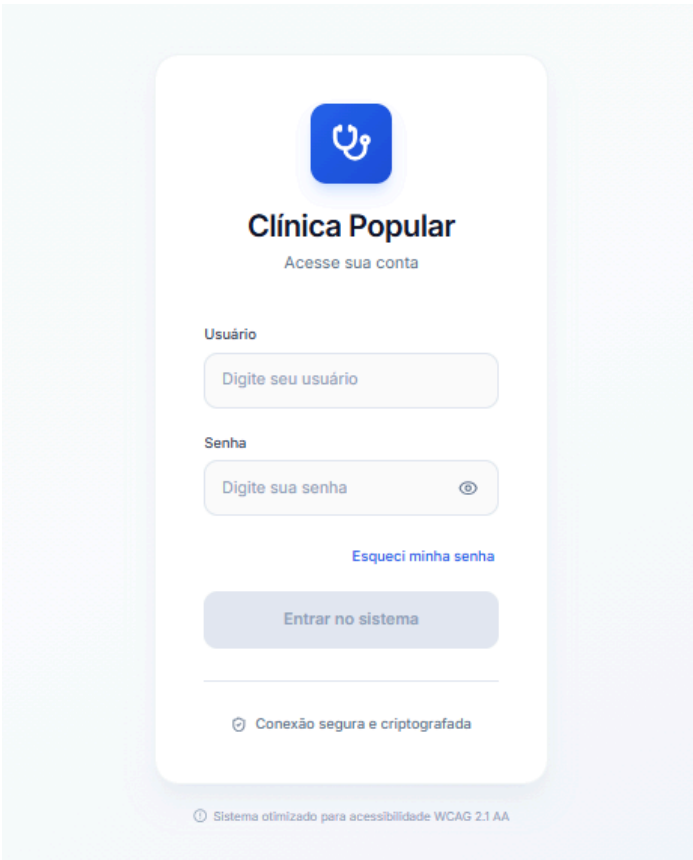
As principais decisões adotadas foram:

- Utilização de hierarquia visual para destacar informações importantes.
- Uso de contraste adequado entre elementos.
- Aplicação de tipografia legível e padronizada.
- Organização consistente dos componentes.

Redução da carga cognitiva dos usuários.

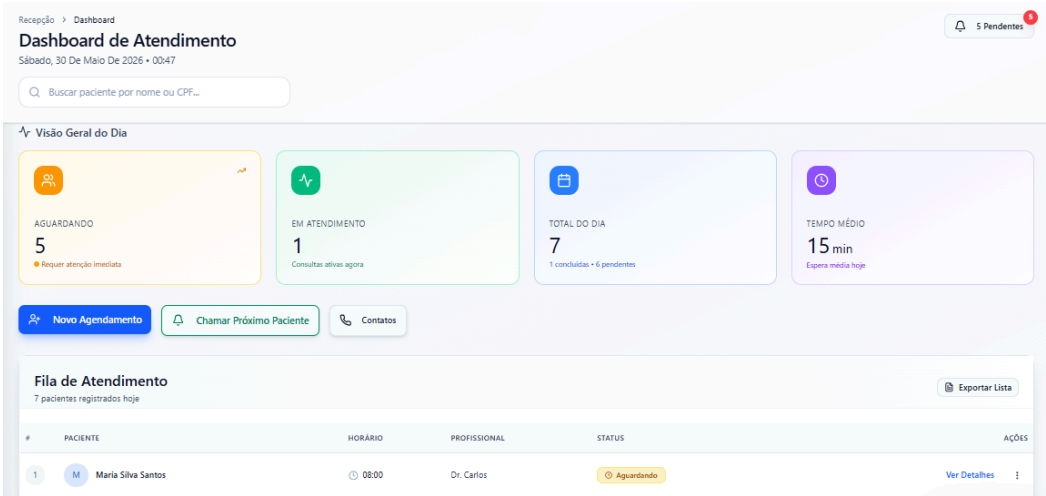
- Utilização de padrões visuais familiares.

Figura 6 - Tela de Login



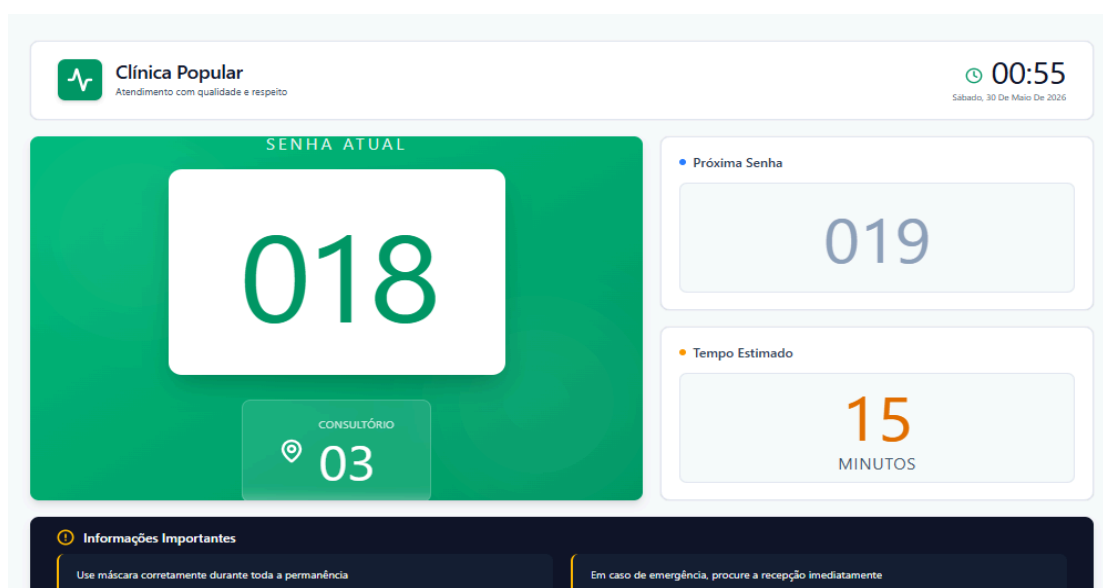
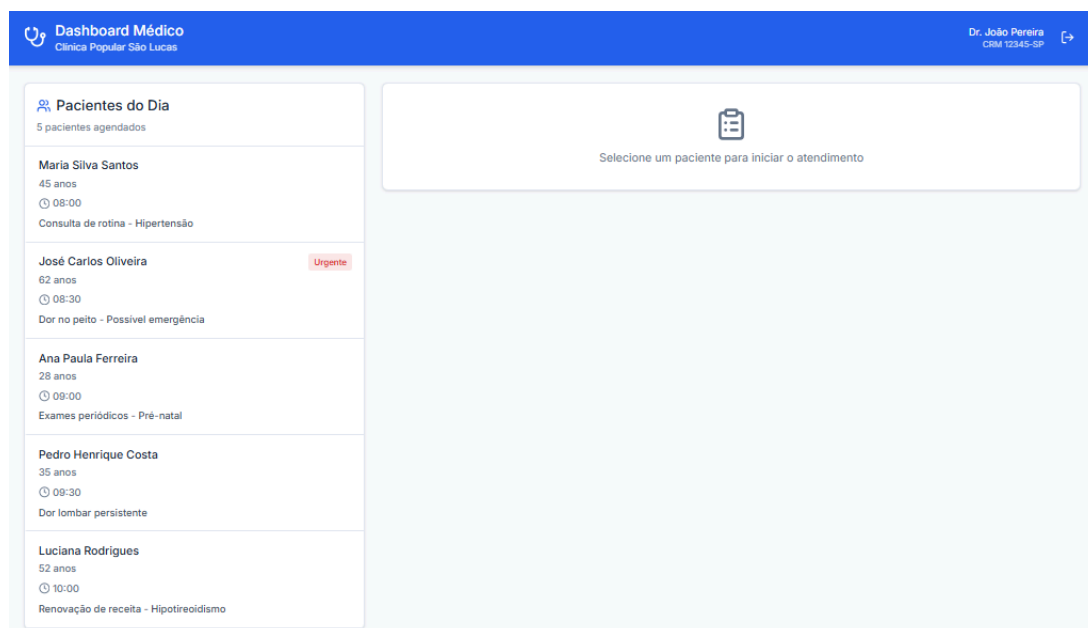
Fonte: Autoria própria

Figura 7 - Dashboard de Atendimento



Fonte: Autoria própria

Figura 8 - Dashboard médico



Fonte: Autoria própria

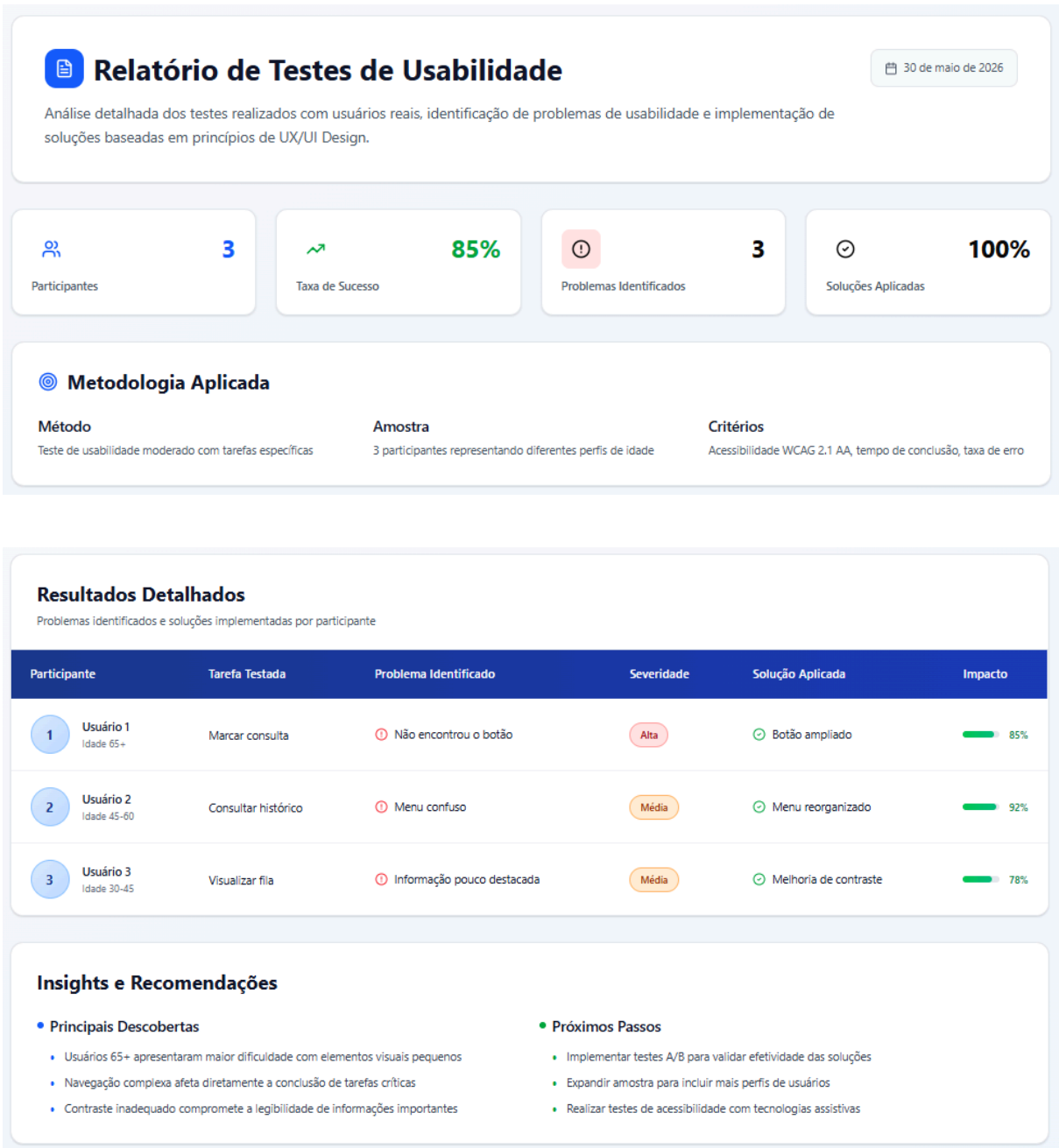
O protótipo desenvolvido permite simular a navegação completa do sistema, possibilitando a avaliação prévia da experiência do usuário antes da implementação da solução.

4.7 Testes de Usabilidade

Com o protótipo concluído, foi realizado um ciclo de testes de usabilidade utilizando usuários simulados. O objetivo foi identificar dificuldades de navegação e oportunidades de melhoria antes da finalização do projeto.

Os participantes receberam tarefas específicas, como realizar login, agendar consultas, consultar informações da fila e acessar o histórico de atendimentos.

Figura 9 - Tela de relatório de usabilidade



Fonte: Autoria própria

Os resultados identificaram oportunidades de melhoria relacionadas à localização de algumas funcionalidades e ao destaque visual de determinados elementos da interface.

Após a análise dos resultados, foram realizados ajustes como:

- Ampliação de botões importantes.
- Reorganização de menus.
- Melhoria do contraste visual.
- Ajuste na disposição de informações da fila.

Essas alterações contribuíram para tornar a experiência mais intuitiva e eficiente.

4.8 Resultados Obtidos

A aplicação dos conceitos de UX e UI Design permitiu desenvolver uma solução mais adequada às necessidades dos usuários da clínica popular.

A utilização das personas contribuiu para compreender diferentes perfis de usuários. As jornadas permitiram identificar pontos críticos da experiência. Os wireframes facilitaram a validação inicial das telas e o protótipo interativo possibilitou testar a navegação antes da implementação do sistema.

Os testes de usabilidade demonstraram que as melhorias realizadas aumentaram a clareza das informações e facilitaram a execução das tarefas pelos usuários simulados.

Dessa forma, conclui-se que a aplicação dos princípios de design centrado no usuário contribuiu significativamente para o desenvolvimento de uma solução mais eficiente, organizada e acessível para o contexto das clínicas populares.

5. MODELAGEM DE BANCO DE DADOS E NOSQL

O banco de dados foi modelado para atender às necessidades de uma clínica popular, permitindo o cadastro de pacientes e profissionais, o gerenciamento de consultas agendadas e o registro de atendimentos realizados. Foram utilizadas chaves primárias e estrangeiras para garantir a integridade dos dados e facilitar a consulta das informações. O modelo segue os princípios da normalização, evitando redundâncias e melhorando a organização dos registros.

5.1 Entidades do banco de dados

5.1.1 Entidade Pacientes

Armazena os dados dos pacientes cadastrados na clínica.

Atributos:

- IdPaciente (Chave Primária)
- Nome
- CPF
- Telefone

5.1.2 Entidade Profissionais

Responsável pelo armazenamento dos dados dos profissionais da saúde.

Atributos:

- IdProfissional (Chave Primária)
- Nome
- Especialidade

5.1.3 Entidade Agendamentos

Controla os agendamentos das consultas entre pacientes e profissionais.

Atributos:

- IdAgendamento (Chave Primária)
- IdPaciente (Chave Estrangeira)
- IdProfissional (Chave Estrangeira)
- DataConsulta
- StatusConsulta

5.1.4 Entidade Atendimentos

Registra os atendimentos realizados pelos profissionais.

Atributos:

- IdAtendimento (Chave Primária)
- IdPaciente (Chave Estrangeira)
- IdProfissional (Chave Estrangeira)
- Diagnóstico

5.2 Relacionamentos entre as tabelas

- Um paciente pode possuir vários agendamentos.
- Um profissional pode possuir vários agendamentos.
- Um paciente pode possuir vários atendimentos.
- Um profissional pode realizar vários atendimentos.

5.3 Regras de Negócio

- Todo agendamento deve estar vinculado a um paciente e a um profissional.
- Um atendimento somente pode ser registrado para pacientes cadastrados.
- Os profissionais devem possuir uma especialidade definida.
- O status da consulta permite acompanhar o andamento do agendamento.

5.4 Script SQL (DDL)

```
CREATE          DATABASE
ClinicaPopular; GO
```

```
USE  ClinicaPopular;
GO
```

```
CREATE TABLE Pacientes (

    IdPaciente INT PRIMARY KEY IDENTITY(1,1),
    Nome VARCHAR(100),

    CPF VARCHAR(14),

    Telefone VARCHAR(20)

);
```

```
CREATE TABLE Profissionais (

    IdProfissional INT PRIMARY KEY IDENTITY(1,1),
    Nome VARCHAR(100),

    Especialidade VARCHAR(50)

);
```



```
CREATE TABLE Agendamentos (
```

```
    IdAgendamento INT PRIMARY KEY IDENTITY(1,1),
```

```
    IdPaciente INT, IdProfissional
```

```
    INT, DataConsulta DATETIME,
```

```
    StatusConsulta VARCHAR(30),
```

```
    FOREIGN KEY (IdPaciente) REFERENCES Pacientes(IdPaciente),
```

```
    FOREIGN KEY (IdProfissional) REFERENCES Profissionais(IdProfissional)
```

```
);
```

```
CREATE TABLE Atendimentos (
```

```
    IdAtendimento INT PRIMARY KEY IDENTITY(1,1),
```

```
    IdPaciente INT, IdProfissional
```

```
    INT, Diagnostico
```

```
    VARCHAR(255),
```

```
    FOREIGN KEY (IdPaciente) REFERENCES Pacientes(IdPaciente),
```

```
    FOREIGN KEY (IdProfissional) REFERENCES Profissionais(IdProfissional)
```

```
);
```

5.5 Inserção de Dados (DML)

```
INSERT INTO Pacientes (Nome, CPF, Telefone)
VALUES
```

```
('João Silva', '12345678900', '(11)99999-1111'),
```

```
('Maria Souza', '98765432100', '(11)98888-2222');
```

```
INSERT INTO Profissionais (Nome, Especialidade)
VALUES
```

```
('Dr. Carlos', 'Clínico Geral'),
```

```
('Dra. Ana', 'Pediatria');
```

```
INSERT INTO Agendamentos
```

```
(IdPaciente, IdProfissional, DataConsulta, StatusConsulta) VALUES
```

```
(1,1,'2026-06-10 09:00','Agendado');
```

5.6 Consultas

5.6.1 Listar pacientes

```
SELECT * FROM Pacientes;
```

5.6.2 Consultas agendadas

```
SELECT
```

```
P.Nome AS Paciente,
```

```
PR.Nome AS Profissional,
```

```
A.DataConsulta
```

```
FROM Agendamentos A
```

```
INNER JOIN Pacientes P ON A.IdPaciente = P.IdPaciente
```

```
INNER JOIN Profissionais PR ON A.IdProfissional = PR.IdProfissional;
```

5.6.3 Histórico de atendimento de um paciente

```
SELECT *
```

```
FROM Atendimentos
```

```
WHERE IdPaciente = 1;
```

6. CONCLUSÃO

Neste projeto foi desenvolvido um ecossistema web para clínicas populares, com a finalidade de auxiliar os profissionais da área e os seus clientes, para cumprir com tal proposta, em engenharia de software definimos quais são os requisitos que o sistema irá atender, sua metodologia, planejamento de sprints, planos de testes e a qualidade de software, em machine learning foi criado os pipelines dos algoritmos de aprendizado de máquina, com UI e UX mostramos como construímos a interface do sistema e com modelagem de banco de dados e nosql projetamos o banco de dados do sistema.

A relevância desse projeto está no fato que a diversos obstáculos presentes dentro de uma clínica popular, desenvolvendo um software como o que foi proposto, conseguimos auxiliar no dia a dia de quem utiliza dos serviços ou trabalha em uma ao resolver alguns desses obstáculos, como por exemplo, ao organizar os prontuários e classificar a probabilidade de atraso de um cliente, podemos criar planejamentos para diminuir o tempo de espera em uma fila na clínica.

Portanto, ao definir e alcançar os objetivos do sistema que foi mostrada a documentação, tivemos a concepção de um software que preza pela usabilidade, confiabilidade e atendimento das necessidades dos usuário, com espaço para melhora, como a criação agentes de IA para atendimentos dos pacientes ou um campo em que eles possam deixar avaliações para termos uma melhora sempre contínua.

REFERÊNCIAS

Clínica Fares (Site institucional). São Paulo. Disponível em:

<https://www.clinicafares.com.br/>. Acesso em: 16 maio 2026.

PORTAL DA SAÚDE. Barueri: Prefeitura de Barueri. Disponível em:

<https://saudeportalcidadao.barueri.sp.gov.br/>. Acesso em: 16 maio 2026.

SOFTDESIGN. Requisitos de software: o que são, tipos e etapas essenciais. 17 jun. 2021. Blog. Disponível em:

<https://www.softdesign.com.br/blog/requisitos-de-software-funcionais-e-nao-funcionais/>. Acesso em: 20 maio 2026.

PUC MINAS GERAIS. Biblioteca Digital. [S.I.]: PUC Minas, [2026?]. PDF. Disponível em:

<https://bib.pucminas.br/pergamumweb/download/7B99FA1B-393A-4DD3-8C94-32052B138AF6.pdf>. Acesso em: 20 maio 2026.

T2 INFORMATIK GMBH. Backlog. In: Smartpedia. [S.I.], [2026?]. Disponível em:

<https://t2informatik.de/en/smartpedia/backlog/>. Acesso em: 20 maio 2026.

GUPY. O que é Sprint: guia completo sobre planejamento, execução e resultados. 8 jan. 2025. Blog. Disponível em: <https://www.gupy.io/blog/sprint>. Acesso em: 20 maio 2026.

BRASIL. Ministério do Esporte. Acesso à informação: LGPD. Brasília, DF: Governo Federal, [2026?]. Disponível em:

<https://www.gov.br/esporte/pt-br/acesso-a-informacao/lgpd>. Acesso em: 20 maio 2026.

GEEKSFORGEEKS. Random Forest Regression in Python. *GeeksforGeeks*, 14 jun. 2019. Atualizado em: 6 abr. 2026. Disponível em: <https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/>. Acesso em: 19 maio 2026.

GEEKSFORGEEKS. Random Forest Classifier using Scikit-learn. *GeeksforGeeks*, 4 set. 2020. Atualizado em: 6 mar. 2026. Disponível em: <https://www.geeksforgeeks.org/random-forest-classifier-using-scikit-learn/>. Acesso em: 19 maio 2026.

Figma. Disponível em: <https://www.figma.com/>. Acesso em: Maio de 2026.

Figma - UX e UI Design. Disponível em: <https://www.figma.com/pt-br/resource-library/diferenca-entre-ui-e-ux/>. Acesso em: Maio de 2026.

Alura. Disponível em: https://www.alura.com.br/artigos/ui-design?srsItd=AfmBOorq48zHYGKpyXRU7WVhibo6W0y4rTd31PjC17_jb1OeU1NVomKI. Acesso em: Maio de 2026.